

1 Types & literals

int i = 42;	32-bit integer
long n = 9_000_000L;	_ separates digits
double d = 3.14;	floating point
decimal m = 19.99m;	exact, for money
int h = 0x2A;	hex; 0b for binary
var x = new List<int>();	type is inferred
int n = default;	0, null or zeroed
const double Pi = 3.14;	compile-time value
readonly DateTime _t;	set in a ctor
public required string Name { get; init; }	must be set · C# 11

2 Strings

(\$"{greeting} {name}!")	interpolation · C# 6
@"C:\Users\Docs"	verbatim string
"""{ "a": 1 }"""	raw literal · C# 11
\$\$"""{ "n": "{x}" }"""	{{ }} is the hole
var u8 = "Hi"u8;	UTF-8 bytes · C# 11
sb.Append(\$"x {i}");	StringBuilder in loops
StringComparison.Ordinal	always pick one

3 Control flow

if (a) X(); else Y();	two-way branch
age >= 18 ? "Adult" : "Kid"	ternary
switch (day) { case DayOfWeek.Sunday: Weekend(); break; default: break; }	break ends each arm
string Kind(object x) => x switch { Circle => "Circle", null => "No shape", _ => "Other" };	switch expression · C# 8
Circle c when c.R > 10	guards one arm

4 Methods & parameters

public int Add(int a, int b) => a + b;	expression-bodied
void Greet(string hi = "Hi")	optional parameter
Greet(hi: "Yo");	named argument
void Bump(ref int n)	caller sees change
bool Try(out int result)	must be assigned
double D(in Point p)	read-only ref
void L(ref readonly Vec3 v)	pass a variable · C# 12
int Sum(params int[] n)	variable-length
params ReadOnlySpan<int> n	no array alloc · C# 13

5 Lambdas & local functions

int Outer(int n) { int T2(int x) => x * 2; return T2(n); }	local function · C# 7
Func<int, int> f = x => x;	returns a value
Action<string> log = Write;	returns void
(x, y = 10) => x + y	default · C# 12

6 Classes, records & structs

public int Age { get; set; }	auto property
public bool Ok => Age >= 18;	computed property
public int Id { get; init; }	init-only · C# 9
class Person(string name)	primary ctor · C# 12
record Person(string Name);	value equality · C# 9
person with { Age = 31 }	copy with changes
record struct Point(int X);	value record · C# 10
sealed class Cache { }	no inheritance
abstract class Shape { }	must be derived

7 Null handling

int? age = null; string? name = null;	nullable value / reference
customer?.Name?.Length	chain stops at null
userName ?? "Anon"	fallback when null
userName ??= "Anon";	set if null · C# 8
x is not null	also x is null
customer?.Order = order;	only if non-null · C# 14
customer?.Balance += 10;	compound · C# 14

8 Loops

for (int i = 0; i < 10; i++)	counted loop
foreach (string n in names)	any IEnumerable
names.Index()	index + item · .NET 9
foreach (var (k, v) in dict)	deconstructs · C# 7
while (cond) { }	may run zero times
break;	also continue;

9 Pattern matching

value is string text	type pattern
p is { Age: >= 30 }	property pattern
x is > 20 and < 30	and or not · C# 9
(x, y) is (> 0, > 0)	tuple pattern
shape is Circle(var r)	positional pattern
arr is [var a, .., var z]	list pattern · C# 11
var (name, _) = person;	discards a value

10 Exceptions

try { Risky(); } catch (IOException ex) { throw; }	add finally to clean up
catch (Ex e) when (IsBad(e))	filter · C# 6
throw;	keeps the stack trace
throw new ArgumentException("Empty.", nameof(x));	name the parameter
ArgumentNullException.ThrowIfNull(x);	guard · .NET 6
class NotFound(int id) : Exception(\$"No {id}") { public int Id => id; }	custom exception
int.TryParse(s, out int n)	for expected misses

11 Async

async Task<string> Get()	Task<T> has a result
await Task.WhenAll(t1, t2);	wait for all
await Task.WhenAny(t1, t2);	first to finish
Task.WhenEach(tasks)	as each lands · .NET 9
await Task.Delay(10, token);	pass the token down
await foreach (var x in Q())	async stream · C# 8
.ConfigureAwait(false)	in library code
async void	event handlers only

12 Code organization

namespace MyApp.Services;	file-scoped · C# 10
global using System.Linq;	whole project · C# 10
using static System.Math;	then just Sqrt(x)
using Pt = (int X, int Y);	alias a type · C# 12
file class Validator { }	this file only · C# 11
public internal protected private protected internal private protected	anywhere / assembly / derived / this type / assembly OR derived / assembly AND derived
public partial string Conn { get; set; }	partial · C# 13
#:package Spectre.Console using Spectre.Console; AnsiConsole.Write("Hi");	dotnet run app.cs · .NET 10

13 Interfaces & generics

public interface ILogger { void Log(string msg); void LogError(string m) => Log(\$"! {m}"); }	default body · C# 8
where T : struct	value type
where T : class	reference type
where T : notnull	never null · C# 8
where T : new()	parameterless ctor
where T : allows ref struct	e.g. Span<T> · C# 13
static T Sum<T>(List<T> xs) where T : INumber<T> => xs.Aggregate(T.Zero, (a, b) => a + b);	generic math · C# 11
static abstract T Create();	on an interface · C# 11

14 Extension members

public static bool IsBlank(this string s) => s.Trim().Length == 0;	classic form
extension<T>(IList<T> src) { public bool IsEmpty => src.Count == 0; }	property · C# 14
extension<T>(List<T>) { public static List<T> Empty => []; }	statics, operators · C# 14

15 Collections

int[] numbers = [1, 2, 3];	collection expr · C# 12
List<string> names = ["A"];	also sets and spans
int[] more = [0, .. nums];	spread operator
Dictionary<string, int> d = new() { ["a"] = 1 };	no collection expr
d.TryGetValue(k, out int v)	no throw on a miss
ReadOnlySpan<int> s = array;	no copy · C# 14
numbers[^1]	last element · C# 8
numbers[1..^1]	range; not on List<T>
queue.Dequeue()	FIFO; stack is LIFO

16 LINQ

xs.Where(n => n > 5)	filter
xs.Select(p => p.Name)	project
xs.OrderBy(p => p.City)	then ThenBy
xs.GroupBy(p => p.City)	gives IGrouping
xs.First(n => n > 5)	also Any, All
query.ToList()	LINQ is lazy
from n in xs where n > 5 select n * 2	query syntax

17 Newer LINQ operators

xs.Chunk(3)	batches · .NET 6
xs.MaxBy(p => p.Age)	also MinBy · .NET 6
xs.CountBy(w => w)	per key · .NET 9
xs.Index()	index + item · .NET 9
people.LeftJoin(orders, p => p.Id, o => o.CustomerId, (p, o) => p.Name)	keeps left rows · .NET 10
numbers.Shuffle()	shuffled · .NET 10

18 New in C# 14 (.NET 10)

extension(List<int> src)	extension members
int Hours { get; set => field = value; }	the field keyword
customer?.Order = value;	null-conditional set
nameof(List<>)	unbound generics
ReadOnlySpan<int> s = array;	first-class spans
(s, out n) => Try(s, out n)	lambda modifiers
partial Settings(string p);	partial ctor, event
public void operator +=(T x)	mutates in place